What you get

- No password prompt on domain PCs
- The user's verified AD account name
- No ADFS, no extra login server
- The server does not need to join the domain

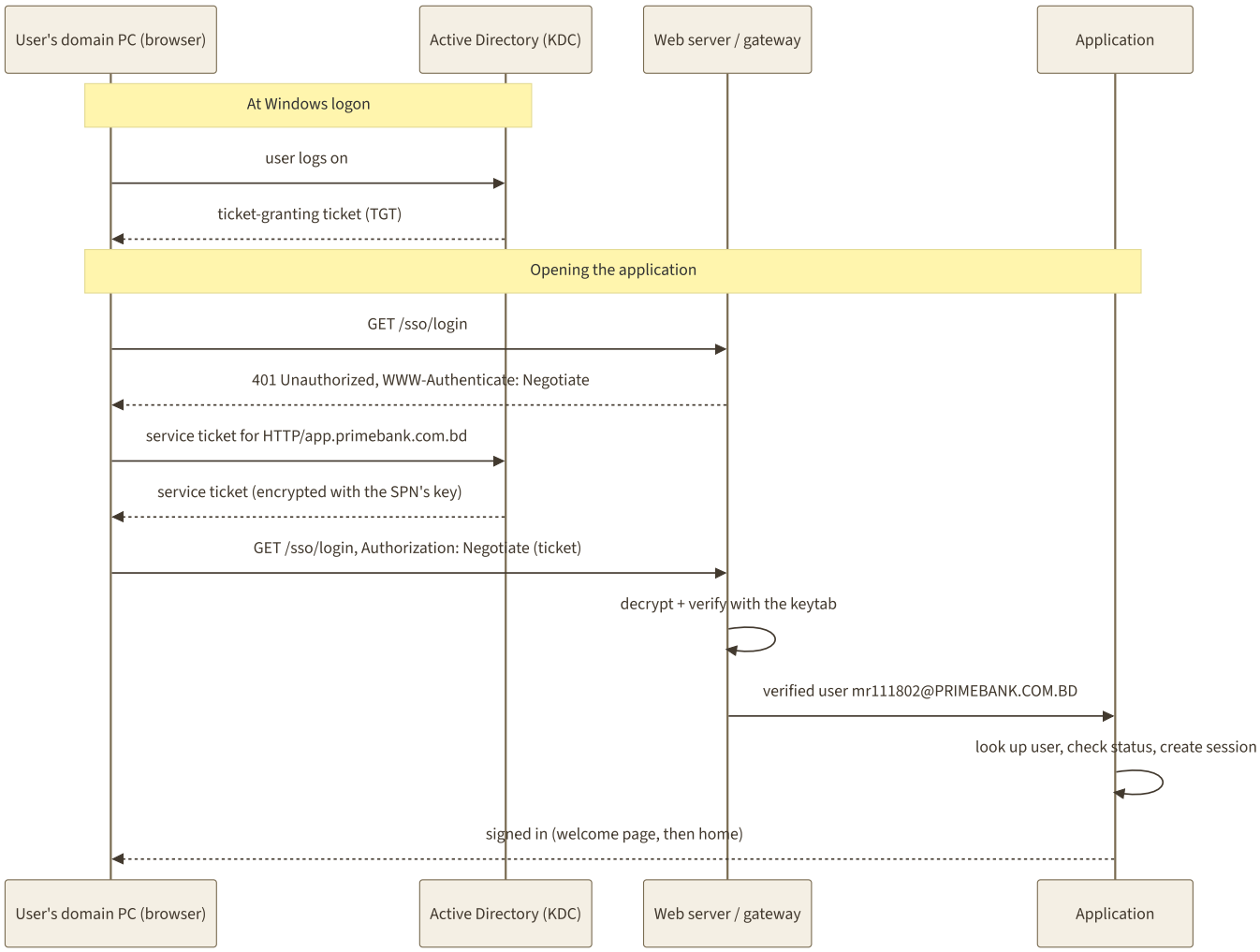
What stays the same

- Your application decides who may use it and with which role
- The normal login page for non-domain PCs and outside access
- Your own sessions, audit log and timeouts

Terms used in this guide

KDC: the Kerberos service on the domain controllers. **SPN**: Service Principal Name, the Kerberos name of your web site, `HTTP/app.primebank.com.bd` . **Keytab**: a file holding the SPN's secret key, used by your server to check tickets. **SPNEGO / Negotiate**: how a ticket travels in HTTP, in the `Authorization: Negotiate …` header.

2. How Kerberos SSO works



1. The server answers the first request with `401` and `WWW-Authenticate: Negotiate` . That is an invitation, not an error.
2. A browser that trusts the site asks AD for a service ticket for `HTTP/<host name>` , built from the host name in the address bar, and repeats the request with it.
3. The server verifies the ticket locally with the keytab. It does not call AD for each login.
4. The verified name (`user@REALM`) is handed to the application, which decides whether that user may use it.
5. A browser without a ticket (non-domain PC, outside the bank) cannot answer the 401; the application must then show its normal login page.

3. What AD must provide

Ask the AD, network and PKI teams for the following, once per application. Replace `app.primebank.com.bd` and `svc-app-web` with your application's names.

ITEM	DETAIL	WHY
DNS **A record**	`app.primebank.com.bd` → server IP. Not a CNAME.	Browsers build the SPN from the name; a CNAME can make them ask for a different SPN.
Service account	`svc-app-web` : password never expires, AES-256 enabled, cannot be delegated, no interactive logon	Owns the SPN and its key.
SPN	`setspn -S HTTP/app.primebank.com.bd PRIMEBANK\svc-app-web`	The Kerberos name of the site. Must exist exactly once in the forest.
Keytab	`ktpass … -crypto AES256-SHA1` , handed over securely	Lets the server verify tickets. Equivalent to a password.
Browser policy (GPO)	Site in the Local intranet zone; Chrome/Edge `AuthServerAllowlist` ; Firefox `network.negotiate-auth.trusted-uris`	Browsers only send tickets to sites they are told to trust.
Proxy bypass	The host bypasses the web proxy, like other intranet sites	Treated as an intranet site; no proxy authentication in the way.
TLS certificate	For the host name, from the bank CA	Tickets and session cookies must travel over HTTPS.
Time sync (NTP)	Server time synchronised with the domain controllers	Kerberos rejects tickets with more than 5 minutes clock skew.

WINDOWS (AD ADMIN)

WINDOWS (AD ADMIN) Copy

```
setspn -S HTTP/app.primebank.com.bd PRIMEBANK\svc-app-web
Set-ADUser svc-app-web -KerberosEncryptionType AES256 -AccountNotDelegated $true -PasswordNever
ktpass -princ HTTP/app.primebank.com.bd@PRIMEBANK.COM.BD -mapuser PRIMEBANK\svc-app-web ^
       -crypto AES256-SHA1 -ptype KRB5_NT_PRINCIPAL -pass * -out app-web.keytab
```

One SPN, one account.

If the same SPN is registered on two accounts, every ticket fails. Always use `setspn -S` (it refuses duplicates) and check with `setspn -Q HTTP/app.primebank.com.bd`. Running `ktpass` again changes the account password: the old keytab stops working immediately.

Worked example: the request sent for Sanction Screening

This is the exact request given to the AD team for Sanction Screening. Copy it for your application and change the host name, IP and account name.

ACTIVITY	REQUEST
DNS creation	A record `thinkai.primebank.com.bd` → `192.168.202.168` . **Note:** it must be an A record, not a CNAME.
Service account creation	Account name `svc-snc-web` . Password set to never expire, and enable **"This account supports Kerb**
SPN registration	`setspn -S HTTP/thinkai.primebank.com.bd PRIMEBANK\svc-snc-web`
Keytab generation	`ktpass -princ HTTP/thinkai.primebank.com.bd@PRIMEBANK.COM.BD -mapuser PRIMEBANK\` Please hand over the keytab file securely once generated.

REQUEST TEXT (READY TO SEND) Copy

```
DNS Creation:
A Record: thinkai.primebank.com.bd [192.168.202.168]
Note: It must be an A record, not a CNAME.

Service Account Creation:
Account Name: svc-snc-web
Settings: Password set to never expire, and enable "This account supports Kerberos AES 256 bit

SPN Registration:
setspn -S HTTP/thinkai.primebank.com.bd PRIMEBANK\svc-snc-web
```

```
Keytab Generation:
ktpass -princ HTTP/thinkai.primebank.com.bd@PRIMEBANK.COM.BD -mapuser PRIMEBANK\svc-snc-web -cr
Please hand over the keytab file securely once generated.
```

The browser policy, proxy bypass, TLS certificate and time synchronisation from the table above are requested the same way, for the same host name.

4. Integration patterns

There are two ways to check the ticket. Both end the same way: your application receives a verified `user@REALM` and applies its own rules.

Recommended

A. Gateway in front

A small Apache server with `mod_auth_gssapi` checks the ticket and forwards only the SSO login request to your application, with the verified user. The application contains no Kerberos code.

- Works for any language
- One proven, maintained component
- Kerberos libraries stay out of the app

Alternative

B. Inside the application

The application's framework checks the ticket itself (Spring Security Kerberos, ASP.NET Core Negotiate, Python `pyspnego`, IIS Windows Authentication).

- No extra component
- Natural on IIS / Windows servers
- Needs Kerberos libraries and the keytab in the app runtime

In both patterns, protect **only one URL** (for example `/sso/login`) with Kerberos. Everything else uses the application's own session, exactly as after a password login.

5. Pattern A: gateway in front (any language)

Apache with `mod_auth_gssapi` (Debian/Ubuntu: `libapache2-mod-auth-gssapi`; RHEL: `mod_auth_gssapi`) offers only Kerberos, verifies the ticket and passes the user on. Your main web server sends only `/sso/login` to it.

APACHE (GATEWAY) Copy

```
Listen 8081
<VirtualHost *:8081>
    # Browser without a ticket: the 401 page sends it to the normal login page.
    ErrorDocument 401 /sso-fallback.html

    <Location "/sso/login">
        AuthType GSSAPI
        AuthName "Single Sign-On"
        GssapiCredStore keytab:/etc/sso/app-web.keytab
        # Kerberos only (never NTLM), never a password prompt, no retry loop
        # after a failure, and keep the full user@REALM name.
        GssapiAllowedMech krb5
```

```
        GssapiBasicAuth Off
        GssapiNegotiateOnce On
        GssapiLocalName Off
        Require valid-user

        # Pass the verified user to the application. Unset first, so a
        # client can never supply these headers itself.
        RequestHeader unset X-SSO-User
        RequestHeader unset X-SSO-Secret
        RequestHeader set   X-SSO-User   "%{REMOTE_USER}s"
        RequestHeader set   X-SSO-Secret "${SSO_GATEWAY_SECRET}"
    </Location>
    ProxyPass        "/sso/login" "http://app-backend:8080/sso/login"
    ProxyPassReverse "/sso/login" "http://app-backend:8080/sso/login"
</VirtualHost>
```

NGINX (MAIN WEB SERVER) Copy

```
# Only the SSO login URL goes to the Kerberos gateway.
location = /sso/login {
    proxy_pass http://sso-gateway:8081;
    proxy_set_header Host $http_host;
    proxy_set_header X-Real-IP $remote_addr;
    # Never let a client send the identity headers through:
    proxy_set_header X-SSO-User "";
    proxy_set_header X-SSO-Secret "";
}
```

The rule that matters most.

The application must accept `X-SSO-User` *only* together with the correct `X-SSO-Secret`, and only on the SSO login URL. Without the secret, anyone who can reach the application could claim to be any user. Some web servers also copy an unverified Basic-auth user name into `REMOTE_USER` (nginx's default `fastcgi_params` does), so never trust `REMOTE_USER` on its own either.

Reading the user in the application

PHP Java Python .NET

PHP

Copy

```
// /sso/login
$secret = trim(file_get_contents('/etc/app/sso-gateway.secret'));
$got    = $_SERVER['HTTP_X_SSO_SECRET'] ?? '';
$user   = $_SERVER['HTTP_X_SSO_USER'] ?? '';
if ($secret === '' || !hash_equals($secret, $got)) {
    header('Location: /login?form=1'); exit;          // not from the gateway
}
if (!preg_match('/^([A-Za-z0-9._-]{1,64})@PRIMEBANK\.COM\.BD$/i', $user, $m)) {
    header('Location: /login?form=1'); exit;          // wrong realm / format
}
$username = strtolower($m[1]);                        // e.g. mr111802
// → section 7: look up the user, check status, start the session
```

6. Pattern B: inside the application

The framework verifies the ticket. The keytab must be readable by the application process (and only by it), and the Kerberos client libraries must be installed (Linux: MIT krb5).

/ETC/KRB5.CONF (LINUX SERVERS)

Copy

```
[libdefaults]
    default_realm = PRIMEBANK.COM.BD
    dns_lookup_kdc = true
    rdns = false
    permitted_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96
```

Java · Spring | .NET | Python | PHP

Spring Security Kerberos (`org.springframework.security.kerberos:spring-security-kerberos-web` , a version matching your Spring Security). Protect only `/sso/login` ; the rest of the app uses its normal login.

JAVA

Copy

```
@Configuration
public class SsoConfig {

    @Bean
    SunJaasKerberosTicketValidator ticketValidator() {
        SunJaasKerberosTicketValidator v = new SunJaasKerberosTicketValidator();
        v.setServicePrincipal("HTTP/app.primebank.com.bd@PRIMEBANK.COM.BD");
        v.setKeyTabLocation(new FileSystemResource("/etc/app/app-web.keytab"));
        return v;
    }

    @Bean
    KerberosServiceAuthenticationProvider kerberosProvider(UserDetailsService appUsers) {
        KerberosServiceAuthenticationProvider p = new KerberosServiceAuthenticationProvider();
```

```
        p.setTicketValidator(ticketValidator());
        p.setUserDetailsService(appUsers);   // your DB lookup (section 7)
        return p;
    }

    @Bean
    AuthenticationManager authenticationManager(KerberosServiceAuthenticationProvider p) {
        return new ProviderManager(p);
    }

    @Bean
    SecurityFilterChain ssoChain(HttpSecurity http, AuthenticationManager am) throws Exception
        SpnegoAuthenticationProcessingFilter spnego = new SpnegoAuthenticationProcessingFilter(
        spnego.setAuthenticationManager(am);
        http.securityMatcher("/sso/login")
            .authorizeHttpRequests(a → a.anyRequest().authenticated())
            .exceptionHandling(e → e.authenticationEntryPoint(new SpnegoEntryPoint("/login?for
            .addFilterBefore(spnego, BasicAuthenticationFilter.class);
        return http.build();
    }
}
// The UserDetailsService receives "mr111802@PRIMEBANK.COM.BD": strip and check the
// realm, load the user from your table, throw UsernameNotFoundException if absent.
```

7. After authentication: the application decides

Kerberos proves *who* the user is. It does not say whether they may use your application. Apply these rules on every SSO login:

1. **Check the realm.** Accept only `@PRIMEBANK.COM.BD`. Anything else goes to the login page.
2. **Normalise the name.** Strip the realm, compare case-insensitively: `MR111802@PRIMEBANK.COM.BD` → `mr111802`.
3. **Look the user up in your own user table.** Not found → "Unauthorized Access – you don't have permission to access this application". Inactive, unverified or pending → "Access denied" with the reason and who to contact.
4. **Create the session exactly as after a password login**: same roles, branch, flags, single-session rule. Renew the session ID at login.
5. **Write the audit log** as for a password login (the bank's inactivity reports rely on the "login success" entry), noting that the method was SSO.

Keeping one shared "finish login" routine for both password and SSO logins guarantees they behave identically.

8. User experience

Login decision flow

What happens when a user opens the login page. Every application implementing SSO should follow the same flow, so users get the same behaviour everywhere.

FLOW

Copy

```
User opens  https://app.primebank.com.bd/login
  |
  ├─ already signed in ──────────────────────────────────────────────► home
  |
  ├─ SSO on + internal network + no ?form / ?logout / ?expired ──► /sso/login  (Kerberos-protec
  |     |
  |     ├─ browser sends a ticket (domain PC) → user, e.g. mr111802@PRIMEBANK.COM.BD
  |     |     → realm checked, name normalised (mr111802)
  |     |     → look up in the application's own user table
  |     |         ├─ not found ──────────────► "Unauthorized Access" + link to the password log
  |     |         ├─ inactive / unverified /
  |     |         |  pending verification ──► "Access denied" + reason, contact IT User admin
  |     |         └─ active ─────────────────► same login as the password form
  |     |                                       (session, roles, branch, single-session rule,
  |     |                                        audit "login success (SSO)")
  |     |                                       → "Single Sign-On enabled" page → home
  |     |
  |     └─ no ticket (non-domain PC / not signed in to AD) ──► login page (password + OTP)
  |
  ├─ from outside / through the internet-facing proxy (DMZ) ──► login page (password + OTP)
  |
  ├─ after Sign out   (?logout=1)  ──► "You are logged out - click here to login again"
  |                                     (no automatic SSO; the link starts a normal sign-in)
  |
  └─ after idle timeout (?expired=1) ─► login page with "Your session expired" (no automatic SS
```

In Sanction Screening the same flow uses `/index.php/signin` and `/index.php/sso/negotiate`; sign-out and timeout both go to a configurable address (`snc_signout_url`).

What the user sees

SITUATION	WHAT THE USER SHOULD SEE
Domain PC, valid user	Opening the site signs in automatically. Optional: a short "Single Sign-On enabled – signed in with your Windows account" page with a progress indicator, then home.
No ticket (non-domain PC, outside the bank)	The normal login page. No error, no browser password pop-up.
Valid AD account, no application account	"Unauthorized Access" with a link to the username/password login (not back to SSO, which would only repeat the message).
Sign out	"You are logged out – click here to login again". The page after sign-out must **not** start SSO automatically, or the user is signed straight back in. Use a flag such as `/login?logout=1`.
Idle timeout	Session ended after N minutes without activity: the login page explains why

	(`?expired=1`), again without automatic SSO.
Wants another account	A "Sign in with username and password" link to `/login?form=1` .

The automatic attempt itself is simple: when an anonymous user opens the login page without `form` , `logout` or `expired` flags, and is on the internal network, redirect to `/sso/login` . Don't try SSO for users coming through the internet-facing reverse proxy.

9. Security requirements

- [ ] Identity accepted only from the verification step: gateway secret checked in constant time, or framework-verified principal. Never from a client-controlled header or a bare `REMOTE_USER` .
- [ ] Kerberos only. No NTLM, no Basic fallback (no password ever travels to the server).
- [ ] Realm checked; user name validated against a strict pattern before any database query.
- [ ] HTTPS on the SSO URL and the whole application; session cookie `Secure` , `HttpOnly` , `SameSite=Lax` .
- [ ] Keytab readable only by the verifying process (e.g. `root:www-data 640`), outside the web root and out of source control; rotated yearly or if exposed.
- [ ] Service account: AES-256 only, "cannot be delegated", no interactive logon, password in the vault.
- [ ] Only the SSO URL is Kerberos-protected; the internal path to it cannot be reached from outside.
- [ ] Successful and refused SSO logins written to the audit log with user, time and source IP.
- [ ] The password login keeps working (and keeps its own AD password check) as fallback and for emergency access.

10. Testing

On a domain PC

WINDOWS COMMAND PROMPT Copy

```
klist purge
:: open https://app.primebank.com.bd in Edge or Chrome, then:
klist | findstr /i HTTP
:: expected: Server: HTTP/app.primebank.com.bd @ PRIMEBANK.COM.BD
```

From a Linux test host (or the server itself)

From a Linux test host (or the server itself):

```shell
kinit someuser@PRIMEBANK.COM.BD
curl -i --negotiate -u : https://app.primebank.com.bd/sso/login     # expect 200/302, not 401
klist -k -e /etc/app/app-web.keytab                                 # principals and AES-256 ke
```

Without the bank's AD

A Samba AD domain controller in a container gives a complete test domain (for example `SNCTEST.LOCAL`): users, a service account, SPN and keytab export with the same steps as the AD team. A client container with `kinit` and `curl --negotiate` then tests every case automatically. Keep it on an internal network only and never join bank PCs to it.

TEST CASE	EXPECTED
No ticket	`401` with `WWW-Authenticate: Negotiate`, then login page
Valid ticket, active user	Signed in; protected pages return 200
Valid ticket, user not in the application	Unauthorized Access page
Valid ticket, inactive user	Access denied with reason
Sign out	Logged-out page; no automatic SSO
Forged identity header without secret	Refused (login page)

11. Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Login page every time; server log shows only `401`, never a retry	Browser does not trust the site	Intranet zone / `AuthServerAllowlist` GPO; check `chrome://policy`
Works in one browser, not another	Policy missing in that browser; Chrome follows CNAMEs	Add the policy; use an A record or Chrome `DisableAuthNegotiateCnameLookup`
Chrome stops trying after one failure	Chrome remembers a failed Negotiate for the site	Restart Chrome after fixing the cause
`klist` shows no `HTTP/…` ticket	SPN missing, wrong name, or duplicated	`setspn -Q HTTP/app…` must return exactly one account
Token in the request starts with	Browser fell back to	Fix the SPN / name; the server must reject NTLM

`TlRMTVNT`	NTLM because Kerberos failed	
Server log: "Decrypt integrity check failed" / `KRB_AP_ERR_MODIFIED`	Keytab does not match the account's current key (password changed, ktpass re-run)	Export a new keytab; check `kvno`
`KRB_AP_ERR_SKEW`	Server clock off by more than 5 minutes	NTP to the domain controllers
Works with the host name, not with the IP	Kerberos needs the host name the SPN was made for	Always use the DNS name
Password pop-up appears	Basic or NTLM offered by the server	Offer Negotiate (Kerberos) only
Signed in, then immediately back to login	Session cookie lost or rejected (domain, path, `Secure` over HTTP)	Check the `Set-Cookie` on the SSO response

12. Go-live checklist

- [ ] AD items delivered: A record, service account, SPN (unique), keytab, GPO, proxy bypass, certificate, NTP.
- [ ] SSO behind a configuration switch, off by default; can be turned off without a deployment.
- [ ] All test cases in section 10 pass in a test domain and on a pilot domain PC.
- [ ] Pilot with a small AD group first; password login still available.
- [ ] Keytab location, owner, permissions and rotation date documented; restart/rollback steps written down.

  Monitoring: SSO successes and refusals visible in the audit log.

13. Reference implementation

Sanction Screening (`https://aml.primebank.com.bd` , PHP on Docker) uses pattern A and can serve as a working example: